SECTION 6 | LECTURE 15

수업 15. 복합적인 위험 검토에 새로운 패턴이 필요할까?

Orchestrator–Workers 도입 판단하기

⚠ 복합 위험 시나리오: 위험 추천 하나에 여러 검토가 필요해진다면?

HYPOTHETICAL AI RECOMMENDATION 운영 배포 롤백 추천 가상 확장안

"Rollback production deployment."

HIGH RISK / APPROVAL REQUIRED

PERSPECTIVE 01

근거 검토(Evidence Review)

최근 배포와 장애 발생 시점 사이의 실질적 인과관계 대조

최근 배포와 장애의 연관성?

PERSPECTIVE 02

품 영향도 검토(Impact Review)

롤백 실행 시 결제, 주문 등 연계 서비스에 미칠 파급 영향 분석

의존 서비스에 미칠 파급 영향?

PERSPECTIVE 03

정책 검토(Policy Review)

배포 동결 시간대 및 서비스별 롤백 승인 정책과의 충돌 검토

서비스별 변경 정책과 충돌?

? **핵심 질문:** 이 질문들을 해결하려면 **새로운 Worker와 Orchestrator가 반드시 필요할까요?**

* 향후 확장 시나리오 가정

검증 및 리뷰 로드맵: 오늘 확인할 것 (기존 구조부터 살펴본다)

STEP 01. IDE REVIEW 현재 코드 책임 확인

Workflow, Quality, Risk, Policy 간의 기존 분리 상태 확인

STEP 02. SCOPE CHECK 추가 요구사항 구분

새 요구사항이 기존 구조로 해결 가능한지 명확히 구분

STEP 03. COMPARISON 아키텍처 대안 비교

기존 구조 확장 vs Orchestrator–Workers 트레이드오프 분석

STEP 04. DECISION 최종 설계 결정

현재 Incident Responder에 지금 도입할 것인지 근거 도출

// CODE REVIEW TARGET RESPONSIBILITIES (NO CODE CHANGES TODAY)

Responsibility Boundary Verification

IncidentResponseWorkflow	Analysis → Action Recommendation → 위험 정책 적용 전체 조율	[전체 흐름]
IncidentQualityControlService	Evaluator-Optimizer 기반 단일 출력 의미 품질 평가 및 개선	[품질 책임]
ActionRiskClassifier & PolicyService	추천 Action의 위험 분류, 의미 중복 제거, 최종 허용 정책 통제	[리스크 책임]

설계 판단 원칙: 오늘은 코드를 추가하지 않고, **현재 코드를 근거로 아키텍처 도입을 판단**합니다.

* Architecture Review Mode

아키텍처 대안 비교: 기존 구조를 확장할까, 별도 검토를 조정할까?

OPTION A. CURRENT EXTENSION

현재 구현 기반

기존 구조 점진적 확장

기존 Workflow 파이프라인 안에서 데이터 조회, 규칙 객체, Java 서비스를 확장하는 방식



필요시 데이터·규칙·서비스 보강으로 충분히 대응 가능

OPTION B. ORCHESTRATOR-WORKERS

가상 확장안 (미구현)

복합 검토 조정 구조

독립적인 전문 검토를 수행하고 충돌·실패 결과를 전담 Orchestrator가 조정하는 방식



독립 데이터와 결과 조정 책임이 명확할 때만 유효

핵심 구분: 책임이 나뉘어 있다는 사실과 **새로운 Orchestrator가 필요하다는 결론은 다릅니다.**

* 두 방안 모두 최종 정책 판단은 Java가 담당

패턴의 본질: Orchestrator–Workers는 어떤 문제를 해결할까?

EVALUATOR-OPTIMIZER

수업 12 패턴

단일 결과물의 품질 평가 & 점진적 개선

```
AI Output → Evaluate → Optimize → Re-Evaluate
[초점] 동일한 결과물의 의미 품질을 단계적으로 개선
```

하나의 출력을 지적 사항에 맞춰 다시 작성하는 반복 루프

ORCHESTRATOR-WORKERS

가상 확장 구조

서로 다른 검토 작업 분할 & 결과 조정

```
Orchestrator → [Evidence / Impact / Policy Workers] → Results
[초점] 독립된 정보 기반의 병렬 작업과 결과 종합·조정
```

독립적인 여러 검토 결과를 수집하고 충돌을 해결하는 조정자

01. INDEPENDENT DATA 독립된 운영 데이터

배포 이력, 의존성 그래프 등 Worker별 고유 데이터 필수

02. SEPARATE CRITERIA 서로 다른 판단 기준

동일 Incident 정보의 단순 프롬프트 복제 호출은 무의미

03. RECONCILIATION 결과 조정의 필요성

검토 결과 간 충돌, 누락, 실패를 중재할 조정 로직 존재

⚠ **도입 전제 조건:** 서로 다른 데이터와 판단 기준이 없다면 **Worker만 늘려도 검토의 신뢰성이 높아지지 않습니다.**

* Worker ≠ Always LLM

⚖ 트레이드오프 분석: 도입하면 얻는 것과 감수할 것은?

BENEFITS

도입 시 얻는 이점

서로 다른 위험을 나누어 검토할 수 있다

역할 분리: 배포 이력, 다른 서비스에 미치는 영향, 정책 위반 여부를 각각 검토

명확한 판단 근거검토 근거 확인: 어떤 검토에서 무엇을 근거로 판단했는지 확인 가능

의견 차이 처리: 검토 결과가 서로 다르거나 확인할 수 없는 경우 별도 규칙으로 처리

처리 과정 추적: 어떤 검토에서 시간이 오래 걸리거나 오류가 발생했는지 확인 가능

여러 검토를 구분하고, 그 결과를 함께 관리할 수 있다.

TRADE-OFFS

도입하면 감수해야 할 것

구조가 복잡해지고 비용이 늘어날 수 있다

추가 데이터 관리: 배포 이력, 서비스 간 연결 정보 등 검토에 필요한 데이터를 수집하고 관리해야 함

검토 결과 관리: Worker별 결과 형식을 정하고, 판단이 서로 다를 때의 처리 규칙이 필요

코드와 테스트 증가: 여러 Worker를 구현하고 테스트해야 하므로 유지보수 부담 증가

LLM 비용과 처리 시간 증가: LLM Worker를 추가하면 Token 사용량과 응답 시간이 늘어날 수 있음

늘어나는 복잡성과 비용만큼 실제로 도움이 되는지 확인해야 한다.

OBSERVATION METRICS: Latency (지연시간 누적) · Tokens (추가 토큰 소모) · Errors (Worker별 부분 실패) · Spans (분기 추적성)

* 도입 시 필수 관찰 항목

최종 설계 결정: 지금은 도입하지 않는다

ENGINEERING DECISION

기존 구조 유지

- ✓ 품질 평가와 개선: 기존 Evaluator-Optimizer가 담당 (수업 12)
- ✓ 위험 분류와 정책 적용: Java 애플리케이션이 담당 (수업 13)
- ✓ 공통 정책 전달: PolicyAdvisor가 여러 LLM 호출에 일관되게 적용 (수업 14)

- ✗ 추가 전문 검토: 배포 이력이나 다른 서비스에 미치는 영향까지 검토해야 하는 요구사항은 아직 없음
- ✗ 여러 범위 결과 조정: 서로 다른 검토 결과의 충돌이나 실패를 별도로 처리할 필요가 아직 없음

현재 Incident Responder는 기존 구조만으로 요구사항을 충족할 수 있다.

FUTURE CRITERIA

🔄 언제 다시 검토해야 할까?

다음 두 가지 요구사항이 실제로 필요해진다면 Orchestrator–Workers 도입을 검토할 수 있습니다.

1. 서로 다른 운영 정보를 활용한 전문 검토가 필요할 때

실제 배포 이력과 장애 발생 시점을 비교하거나, 특정 작업이 다른 서비스에 미치는 영향까지 검토해야 하는 경우

2. 여러 검토 결과를 함께 조정해야 할 때

검토 결과가 서로 다르거나 일부 검토가 실패했을 때, 결과를 종합하고 처리 방법을 결정하는 별도의 조정 역할이 필요한 경우

단, 이 경우에도 기존 구조로 해결할 수 있는지 먼저 확인합니다.

> **설계 결론: 새로운 패턴이 항상 필요한 것은 아닙니다. 현재 문제를 해결하는 데 필요한 구조만 사용합니다.**

🏁 오늘 정리 & 다음 수업 안내

TODAY SUMMARY

Architecture Review

⚖️ Orchestrator–Workers 도입이 필요한지 검토

현재 구조 확인: 품질 평가, 위험 분류, 정책 적용의 역할이 이미 나뉘어 있는지 확인

새로운 요구사항 검토: 배포 이력이나 다른 서비스에 미치는 영향까지 검토해야 하는 상황 살펴보기

설계 방법 비교: 기존 Workflow를 확장하는 방법과 Orchestrator–Workers를 도입하는 방법 비교

최종 결정: 현재는 새로운 패턴을 도입하지 않고 기존 구조 유지

> **핵심 메시지:** 새로운 패턴을 적용하는 것보다 현재 문제에 맞는 구조를 선택하는 것이 중요합니다.

UP NEXT LECTURE (PART 1 FINALE)

🗺️ Part 1 최종 완성 (Final Control Map)

지금까지 배운 **흐름·비용·품질·리스크 통제**가 Incident Responder의 요청 처리 과정에서 어디에 적용되는지 하나의 그림으로 정리합니다.

- 흐름 통제: 요청 경로 선택과 실행 과정 추적
- 비용 통제: LLM에 전달하는 정보량 제한
- 품질 통제: AI 응답 검증과 품질 개선
- 위험 통제: 위험 추천 분류와 정책 적용

최종 목표: **네 가지 통제 지점이 어디에 있고, 어떻게 연결되는지 한눈에 확인합니다.**

Move to Final Control Map ›